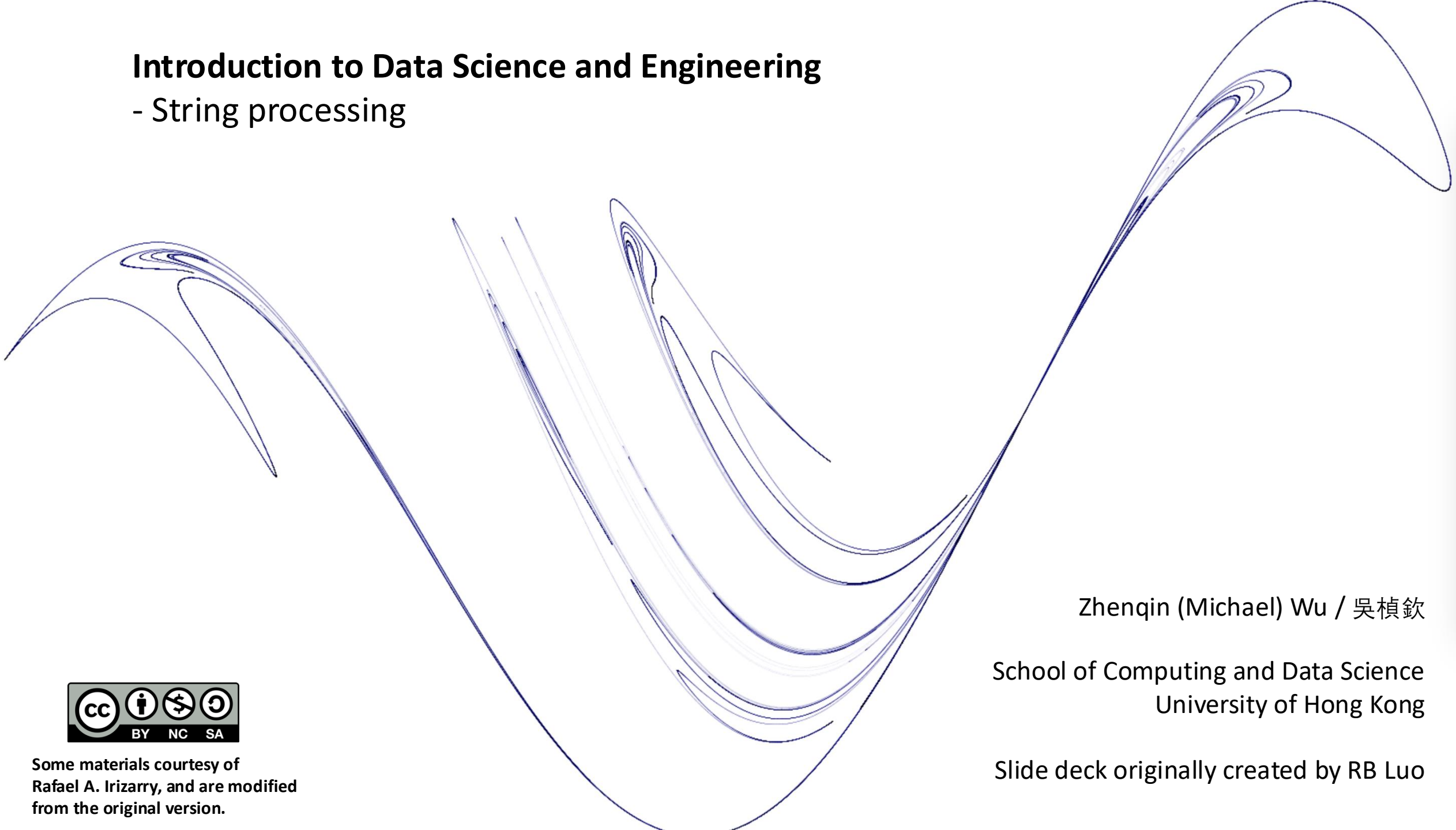
Introduction to Data Science and Engineering

- String processing



Zhenqin (Michael) Wu / 吳楨欽

School of Computing and Data Science
University of Hong Kong

Slide deck originally created by RB Luo

Some materials courtesy of Rafael A. Irizarry, and are modified from the original version.

In this lecture

- String processing with `stringr`
- Regular Expression
- Parsing date and time

In this lecture

- **String processing with `stringr`**
- Regular Expression
- Parsing date and time

String processing in data wrangling

- One of the most common data wrangling challenges
  - Extracting numbers from strings: “1000”, “1,000,000”, “1,00,000”, “1e5”, “250930”, “25-09-30”, “(342)346-0452”
  - Aligning capitalization
  - Handling missing values: “NA”, “N/A”, “Unknown”, “Unkonwn”, “---”
  - Non-ASCII: “café”, “吴 Zhenqin”

- How to deal with these cases:
  - extract numbers;
  - remove unwanted characters;
  - find and replace characters;
  - extract specific parts of strings;
  - …

`stringr`

- Part of `tidyverse`;
- All `stringr` functions take a character vector as first argument, thus they can be used with piping;
- All `stringr` operations are vectorized.

Work with strings with stringr : : **CHEAT SHEET**

The **stringr** package provides a set of internally consistent tools for working with character strings, i.e. sequences of characters surrounded by quotation marks.

Detect Matches

str_detect(string, **pattern**) Detect the presence of a pattern match in a string.
str_detect(fruit, "a")

str_which(string, **pattern**) Find the indexes of strings that contain a pattern match.
str_which(fruit, "a")

str_count(string, **pattern**) Count the number of matches in a string.
str_count(fruit, "a")

str_locate(string, **pattern**) Locate the positions of pattern matches in a string. Also **str_locate_all**. *str_locate(fruit, "a")*

Subset Strings

str_sub(string, start = 1L, end = -1L) Extract substrings from a character vector.
str_sub(fruit, 1, 3); str_sub(fruit, -2)

str_subset(string, **pattern**) Return only the strings that contain a pattern match.
str_subset(fruit, "b")

str_extract(string, **pattern**) Return the first pattern match found in each string, as a vector. Also **str_extract_all** to return every pattern match. *str_extract(fruit, "[aeiou]")*

str_match(string, **pattern**) Return the first pattern match found in each string, as a matrix with a column for each ( ) group in pattern. Also **str_match_all**.
str_match(sentences, "(a|the) ([^]+)")

Manage Lengths

str_length(string) The width of strings (i.e. number of code points, which generally equals the number of characters). *str_length(fruit)*

str_pad(string, width, side = c("left", "right", "both"), pad = " ") Pad strings to constant width. *str_pad(fruit, 17)*

str_trunc(string, width, side = c("right", "left", "center"), ellipsis = "...") Truncate the width of strings, replacing content with ellipsis.
str_trunc(fruit, 3)

str_trim(string, side = c("both", "left", "right")) Trim whitespace from the start and/or end of a string. *str_trim(fruit)*

Mutate Strings

str_sub() **<-** value. Replace substrings by identifying the substrings with str_sub() and assigning into the results.
str_sub(fruit, 1, 3) <- "str"

str_replace(string, **pattern**, replacement) Replace the first matched pattern in each string. *str_replace(fruit, "a", "-")*

str_replace_all(string, **pattern**, replacement) Replace all matched patterns in each string. *str_replace_all(fruit, "a", "-")*

str_to_lower(string, locale = "en")[1] Convert strings to lower case.
str_to_lower(sentences)

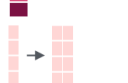
str_to_upper(string, locale = "en")[1] Convert strings to upper case.
str_to_upper(sentences)

str_to_title(string, locale = "en")[1] Convert strings to title case. *str_to_title(sentences)*

Join and Split

str_c(..., sep = "", collapse = NULL) Join multiple strings into a single string.
str_c(letters, LETTERS)

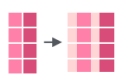
str_c(..., sep = "", **collapse = NULL**) Collapse a vector of strings into a single string.
str_c(letters, collapse = "")



str_dup(string, times) Repeat strings times times. *str_dup(fruit, times = 2)*

str_split_fixed(string, **pattern**, n) Split a vector of strings into a matrix of substrings (splitting at occurrences of a pattern match). Also **str_split** to return a list of substrings.
str_split_fixed(fruit, " ", n=2)

glue::**glue**(..., .sep = "", .envir = parent.frame(), .open = "{", .close = "}") Create a string from strings and {expressions} to evaluate. *glue::glue("Pi is {pi}")*



glue::**glue_data**(.x, ..., .sep = "", .envir = parent.frame(), .open = "{", .close = "}") Use a data frame, list, or environment to create a string from strings and {expressions} to evaluate. *glue::glue_data(mtcars, "{rownames(mtcars)} has {hp} hp")*

Order Strings

str_order(x, decreasing = FALSE, na_last = TRUE, locale = "en", numeric = FALSE, ...)[1] Return the vector of indexes that sorts a character vector. *x[str_order(x)]*

str_sort(x, decreasing = FALSE, na_last = TRUE, locale = "en", numeric = FALSE, ...)[1] Sort a character vector.
str_sort(x)

Helpers

```
apple
banana
pear
```

str_conv(string, encoding) Override the encoding of a string. *str_conv(fruit,"ISO-8859-1")*

```
apple
banana
pear
```

str_view(string, **pattern**, match = NA) View HTML rendering of first regex match in each string. *str_view(fruit, "[aeiou]")*

str_view_all(string, **pattern**, match = NA) View HTML rendering of all regex matches.
str_view_all(fruit, "[aeiou]")

str_wrap(string, width = 80, indent = 0, exdent = 0) Wrap strings into nicely formatted paragraphs. *str_wrap(sentences, 20)*

See also: https://rstudio.github.io/cheatsheets/html/strings.html
https://github.com/rstudio/cheatsheets/blob/main/strings.pdf

Case study 1: US murders data

This is where we ended from web scraping

```
> url <- "https://en.wikipedia.org/w/index.php?title=Gun_violence
_in_the_United_States_by_state&direction=prev&oldid=810166167"
> tabs <- read_html(url) |> html_elements("table")
> murders_raw <- tabs[[1]] |>
+   html_table() |>
+   setNames(c("state", "population", "total", "murder_rate"))
> murders_raw
# A tibble: 51 × 4
   state                population total murder_rate
   <chr>                <chr>      <chr>       <dbl>
 1 Alabama              4,853,875  348           7.2
 2 Alaska               737,709    59            8
 3 Arizona              6,817,565  309           4.5
 4 Arkansas             2,977,853  181           6.1
 5 California           38,993,940 1,861         4.8
 6 Colorado             5,448,819  176           3.2
 7 Connecticut          3,584,730  117           3.3
 8 Delaware             944,076    63            6.7
 9 District of Columbia 670,377    162          24.2
10 Florida              20,244,914 1,041         5.1
# ℹ 41 more rows
# ℹ Use `print(n = ...)` to see more rows
```

`as.numeric` doesn't work due to thousand comma separators

```
> as.numeric(murders_raw$population)
 [1] NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA
[23] NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA
[45] NA NA NA NA NA NA NA
Warning message:
NAs introduced by coercion
```

we use `str_detect` to detect which columns contain comma(s)

```
> str_detect(murders_raw$state, ",") |> any()
[1] FALSE
> str_detect(murders_raw$population, ",") |> any()
[1] TRUE
> str_detect(murders_raw$total, ",") |> any()
[1] TRUE
> str_detect(murders_raw$murder_rate, ",") |> any()
[1] FALSE
```

Just in case you need it: https://en.wikipedia.org/w/index.php?title=Gun_violence_in_the_United_States_by_state&direction=prev&oldid=810166167

Case study 1: US murders data

```
> test_1 <- murders_raw |>
+   pull(population) |>
+   str_replace_all(",", "") |>
+   as.numeric()
> test_1
 [1]  4853875   737709  6817565  2977853 38993940  5448819
 [7]  3584730   944076   670377 20244914 10199398  1425157
[13]  1652828 12839047  6612768  3121997  2906721  4424611
[19]  4668960  1329453  5994983  6784240  9917715  5482435
[25]  2989390  6076204  1032073  1893765  2883758  1330111
[31]  8935421  2080328 19747183 10035186   756835 11605090
[37]  3907414  4024634 12791904  1055607  4894834   857919
[43]  6595056 27429639  2990632   626088  8367587  7160290
[49]  1841053  5767891   586107
> murders_raw$population
 [1] "4,853,875"  "737,709"    "6,817,565"  "2,977,853"
 [5] "38,993,940" "5,448,819"  "3,584,730"  "944,076"
 [9] "670,377"    "20,244,914" "10,199,398" "1,425,157"
[13] "1,652,828"  "12,839,047" "6,612,768"  "3,121,997"
[17] "2,906,721"  "4,424,611"  "4,668,960"  "1,329,453"
[21] "5,994,983"  "6,784,240"  "9,917,715"  "5,482,435"
[25] "2,989,390"  "6,076,204"  "1,032,073"  "1,893,765"
[29] "2,883,758"  "1,330,111"  "8,935,421"  "2,080,328"
[33] "19,747,183" "10,035,186" "756,835"    "11,605,090"
[37] "3,907,414"  "4,024,634"  "12,791,904" "1,055,607"
[41] "4,894,834"  "857,919"    "6,595,056"  "27,429,639"
[45] "2,990,632"  "626,088"    "8,367,587"  "7,160,290"
[49] "1,841,053"  "5,767,891"  "586,107"
```

Use `str_replace_all` to remove patterns

When processing strings:

- One column at a time
- Sanity check, and double check

Case study 1: US murders data

```
> test_1 <- murders_raw |>
+   pull(population) |>
+   str_replace_all(",", "") |>
+   as.numeric()
> murders_processed_1 <- murders_raw |>
+   mutate(population = test_1)
> murders_processed_1
# A tibble: 51 × 4
   state                population total murder_rate
   <chr>                     <dbl> <chr>        <dbl>
 1 Alabama                 4853875 348            7.2
 2 Alaska                   737709 59             8
 3 Arizona                 6817565 309            4.5
 4 Arkansas                2977853 181            6.1
 5 California             38993940 1,861          4.8
 6 Colorado                5448819 176            3.2
 7 Connecticut             3584730 117            3.3
 8 Delaware                 944076 63             6.7
 9 District of Columbia     670377 162           24.2
10 Florida                20244914 1,041          5.1
# i 41 more rows
# i Use `print(n = ...)` to see more rows
```

Once you are sure that this pipeline works as expected, proceed with `mutate`

Case study 1: US murders data

Removing comma from numbers is a common operation, we can use `parse_number` instead.

```
> test_2 <- murders_raw |>
+   pull(population) |>
+   parse_number()
> all(test_1 == test_2)
[1] TRUE
```

How about other cases:

- “1000”, “1,000,000”, “1,00,000”, “1e5”, “250930”
- “25-09-30”, “(342)346-0452”: should `parse_number` work on these cases? Does it work? Any takeaway?

Case study 1: US murders data

```
> filename <- system.file("extdata/murders.csv", package = "dslabs")
> lines <- readLines(filename)
> lines |> head()
[1] "state,abb,region,population,total" "Alabama,AL,South,4779736,135"
[3] "Alaska,AK,West,710231,19"         "Arizona,AZ,West,6392017,232"
[5] "Arkansas,AR,South,2915918,93"     "California,CA,West,37253956,1257"
```

A common operation is string splitting.

Here we are using the base R function `readLines` to read the source csv file of the `murders` dataset

```
> x <- str_split(lines, ",")
> class(x)
[1] "list"
> x |> head(2)
[[1]]
[1] "state"      "abb"        "region"     "population" "total"

[[2]]
[1] "Alabama" "AL"      "South"   "4779736" "135"
```

Note that this file contains a table, using commas as separator. We can use the method `str_split` to split each line/row to multiple entries.

`x` here is a list

```
> col_names <- x[[1]]
> x <- x[-1]
```

Note that the first line/row contains column names

Case study 1: US murders data

```
> x <- str_split(lines, ",", simplify = TRUE)
> class(x)
[1] "matrix" "array"
> col_names <- x[1,]
> x <- x[-1,]
> colnames(x) <- col_names
> x |> as_data_frame() |> head(3)
# A tibble: 3 × 5
  state   abb   region population total
  <chr>   <chr> <chr>  <chr>      <chr>
1 Alabama AL    South  4779736    135
2 Alaska  AK    West   710231     19
3 Arizona AZ    West   6392017    232
```

`str_split(… simplify = TRUE)` returns a character matrix

`simplify`	• `FALSE` (the default): returns a list of character vectors. • `TRUE`: returns a character matrix.

`mutate_all`: apply the same function to all columns

`parse_guess`: automatically detect and set data types

```
> x |> as_data_frame() |> mutate_all(parse_guess) |> head(3)
# A tibble: 3 × 5
  state   abb   region population total
  <chr>   <chr> <chr>       <dbl> <dbl>
1 Alabama AL    South     4779736   135
2 Alaska  AK    West       710231    19
3 Arizona AZ    West      6392017   232
```

https://www.rdocumentation.org/packages/readr/versions/2.1.3/topics/parse_guess

Case study 2: Self-reported heights

`data(reported_heights)` is the raw data for `data(heights)`

```
> data(reported_heights)
> reported_heights
             time_stamp    sex                height
1   2014-09-02 13:40:36   Male                    75
2   2014-09-02 13:46:59   Male                    70
3   2014-09-02 13:59:20   Male                    68
4   2014-09-02 14:51:53   Male                    74
5   2014-09-02 15:16:15   Male                    61
6   2014-09-02 15:16:16 Female                    65
7   2014-09-02 15:16:19 Female                    66
8   2014-09-02 15:16:21 Female                    62
9   2014-09-02 15:16:21 Female                    66
10  2014-09-02 15:16:22   Male                    67
11  2014-09-02 15:16:22   Male                    72
12  2014-09-02 15:16:23   Male                     6
13  2014-09-02 15:16:23   Male                    69
14  2014-09-02 15:16:26   Male                    68
15  2014-09-02 15:16:26   Male                    69
16  2014-09-02 15:16:26   Male                    66
17  2014-09-02 15:16:27   Male                    75
18  2014-09-02 15:16:27 Female                    64
19  2014-09-02 15:16:27 Female                    60
20  2014-09-02 15:16:28   Male                    67
21  2014-09-02 15:16:28   Male                    66
22  2014-09-02 15:16:28   Male                 5' 4"
```

"These heights were obtained using a google survey in which students were asked to enter their heights. They could enter anything, but the instructions asked for height in inches, a number. We compiled 1,095 submissions, but unfortunately the column vector with the reported heights had several non-numeric entries and as a result became a character vector"

```
> class(reported_heights$height)
[1] "character"
```

Case study 2: Self-reported heights

If we try to convert it into numbers, we get a warning

```
> x <- as.numeric(reported_heights$height)
Warning message:
NAs introduced by coercion
```

81 rows end up `NA`s

```
> sum(is.na(x))
[1] 81
```

```
> reported_heights |>
+ mutate(new_height = as.numeric(height)) |>
+ filter(is.na(new_height)) |>
+ head(10)
           time_stamp    sex                 height new_height
1  2014-09-02 15:16:28   Male                  5' 4"         NA
2  2014-09-02 15:16:37 Female                 165cm         NA
3  2014-09-02 15:16:52   Male                   5'7         NA
4  2014-09-02 15:16:56   Male                 >9000         NA
5  2014-09-02 15:16:56   Male                  5'7"         NA
6  2014-09-02 15:17:09 Female                  5'3"         NA
7  2014-09-02 15:18:00   Male 5 feet and 8.11 inches         NA
8  2014-09-02 15:19:48   Male                  5'11         NA
9  2014-09-04 00:46:45   Male                 5'9''         NA
10 2014-09-04 10:29:44   Male                5'10''         NA
```

List entries that are not successfully converted by using `filter` to keep only `NA` rows

We immediately found what's happening, many reporting in feet and inches, not just inches as requested

Case study 2: Self-reported heights

Even for the converted numbers, are they all correct?

```
> reported_heights |>
+ mutate(new_height = as.numeric(height)) |>
+ filter(new_height > 84 | new_height < 50)
             time_stamp    sex height new_height
1   2014-09-02 15:16:23   Male      6       6.00
2   2014-09-02 15:16:32 Female    5.3       5.30
3   2014-09-02 15:16:41   Male    511     511.00
4   2014-09-02 15:16:46   Male      6       6.00
5   2014-09-02 15:16:50 Female      2       2.00
6   2014-09-02 15:18:14 Female   5.25       5.25
7   2014-09-03 21:43:00   Male    5.5       5.50
8   2014-09-03 23:55:37   Male  11111   11111.00
9   2014-09-04 05:15:28 Female      6       6.00
10  2014-09-04 06:31:03   Male    6.5       6.50
11  2014-09-04 09:24:41 Female    150     150.00
12  2014-09-04 14:23:19   Male  103.2     103.20
13  2014-09-06 17:01:32   Male    5.8       5.80
14  2014-09-06 21:32:15   Male     19      19.00
15  2014-09-06 22:38:40 Female      5       5.00
16  2014-09-07 14:35:35 Female    5.6       5.60
17  2014-09-07 16:33:46   Male    175     175.00
18  2014-09-07 18:18:31   Male    177     177.00
```

```
> reported_heights |>
+ mutate(new_height = as.numeric(height)) |>
+ filter(new_height > 84 | new_height < 50) |> dim()
[1] 211   4
```

There are 211 rows that are either:

- Taller than 84 inches (213cm)
- Shorter than 50 inches (127cm)

Case study 2: Self-reported heights

We assemble these sanity checks in a function so to be used again during string processing

```
> not_inches <- function(x, smallest = 50, tallest = 84){
+     inches <- suppressWarnings(as.numeric(x))
+     ind <- is.na(inches) | inches < smallest | inches > tallest
+     ind
+ }
```

```
> problems <- reported_heights |>
+ filter(not_inches(height)) |>
+ pull(height)
> length(problems)
[1] 292
```

We apply this function to the raw data and find 292 problematic entries

Next, we will:

1. Look at some problematic cases;
2. Identify common mistake patterns;
3. Try finding ways to correct or remove the mistakes

Case study 2: Self-reported heights

Common mistake patterns:

1. x'y / x' y'' / x’y”: feet’inches’’
2. x.y: feet.inches
3. Centimeters
4. x feet y inches

Mistakes/artifacts/outliers:

- >9000
- 649,606
- 11111
- …

```
> paste(problems, collapse="  ")
[1] "6  5' 4\"  5.3  165cm  511  6  2  5'7  >9000  5'7\"  5'3\"  5 feet and
8.11 inches  5.25  5'11  5.5  11111  5'9''  6  6.5  150  5'10''  103.2  5.8
19  5  5.6  175  177  300  5,3  6'  6  5.9  6,8  5' 10  5.5  178  163  6.2  1
75  Five foot eight inches  6.2  5.8  5.1  178  165  5.11  5'5\"  165  180
5'2\"  5.75  169  5,4  7  5.4  157  6.1  169  5'3  5.6  214  183  5.6  6  162
178  180  5'10''  170  5'3''  178  0.7  190  5.4  184  5'7''  5.9  5'12  5.6
5.6  184  6  167  2'33  5'11  5'3\"  5.5  5.2  180  5.5  5.5  6.5  5,8  180
183  170  5'6''  172  612  5.11  168  5'4  1,70  172  87  5.5  176  5'7.5''
5'7.5''  111  5'2\"  173  174  176  175  5' 7.78\"  6.7  12  6  5.1  5.6  5.5
yyy  5.2  5'5  5'8  5'6  5 feet 7inches  89  5.6  5.7  183  172  34  25  6
5.9  168  6.5  170  175  6  22  5.11  684  6  1  1  6*12  5 .11  87  162  165
184  6  173  1.6  172  170  5.7  5.5  174  170  160  120  120  23  192  5 11
167  150  1.7  174  5.8  6  5'4  5'8\"  5'5  5.8  5.1  5.11  5.7  5'7  5'6
5'11\"  5'7\"  5'7  172  5'8  180  5' 11\"  5  180  180  6'1\"  5.9  5.2  5.5
69\"  5' 7\"  5'10''  5.51  5'10  5'10  5ft 9 inches  5 ft 9 inches  5'2  5'1
1  5.8  5.7  167  168  6  6.1  5'11''  5.69  178  182  164  5'8\"  185  6  86
5.7  708,661  5.25  5.5  5 feet 6 inches  5'10''  172  6  5'8  160  6'3\"  64
9,606  10000  5.1  152  1  180  728,346  175  158  173  164  6 04  169  0  18
5  168  5'9  169  5'5''  174  6.3  179  5'7\"  5.5  6  6  170  6  172  158  1
00  159  190  5.7  170  158  6'4\"  180  5.57  5'4  210  88  6  162  170 cm
5.7  170  157  186  170  7,283,465  5  5  34  161  5'6  5'6"
```

Case study 2: Self-reported heights

We will need to

- Identify entries that satisfy the common mistake pattern (1) & (2)
- Extract feet, inches from these entries and recalculate them
- Identify centimeter cases as in (3)
- Check again problematic cases

Do we just `str_detect` and `str_replace` using the fixed patterns?

Introducing regular expressions!

```
> paste(problems, collapse="  ")
[1] "6  5' 4\"  5.3  165cm  511  6  2  5'7  >9000  5'7\"  5'3\"  5 feet and
8.11 inches  5.25  5'11  5.5  11111  5'9''  6  6.5  150  5'10''  103.2  5.8
19  5  5.6  175  177  300  5,3  6'  6  5.9  6,8  5' 10  5.5  178  163  6.2  1
75  Five foot eight inches  6.2  5.8  5.1  178  165  5.11  5'5\"  165  180
5'2\"  5.75  169  5,4  7  5.4  157  6.1  169  5'3  5.6  214  183  5.6  6  162
178  180  5'10''  170  5'3''  178  0.7  190  5.4  184  5'7''  5.9  5'12  5.6
5.6  184  6  167  2'33  5'11  5'3\"  5.5  5.2  180  5.5  5.5  6.5  5,8  180
183  170  5'6''  172  612  5.11  168  5'4  1,70  172  87  5.5  176  5'7.5''
5'7.5''  111  5'2\"  173  174  176  175  5' 7.78\"  6.7  12  6  5.1  5.6  5.5
yyy  5.2  5'5  5'8  5'6  5 feet 7inches  89  5.6  5.7  183  172  34  25  6
5.9  168  6.5  170  175  6  22  5.11  684  6  1  1  6*12  5 .11  87  162  165
184  6  173  1.6  172  170  5.7  5.5  174  170  160  120  120  23  192  5 11
167  150  1.7  174  5.8  6  5'4  5'8\"  5'5  5.8  5.1  5.11  5.7  5'7  5'6
5'11\"  5'7\"  5'7  172  5'8  180  5' 11\"  5  180  180  6'1\"  5.9  5.2  5.5
69\"  5' 7\"  5'10''  5.51  5'10  5'10  5ft 9 inches  5 ft 9 inches  5'2  5'1
1  5.8  5.7  167  168  6  6.1  5'11''  5.69  178  182  164  5'8\"  185  6  86
5.7  708,661  5.25  5.5  5 feet 6 inches  5'10''  172  6  5'8  160  6'3\"  64
9,606  10000  5.1  152  1  180  728,346  175  158  173  164  6 04  169  0  18
5  168  5'9  169  5'5''  174  6.3  179  5'7\"  5.5  6  6  170  6  172  158  1
00  159  190  5.7  170  158  6'4\"  180  5.57  5'4  210  88  6  162  170 cm
5.7  170  157  186  170  7,283,465  5  5  34  161  5'6  5'6"
```

A quick note on escaping characters

To define strings in R, we can use either double quotes or single quotes

Backticks is used when you have special characters in the variable name.

To use a double quote in double quotes:

- Use backslash as an escape
- Mix use of ‘ ’ and “ ”

My suggestion is to always use backslash, there are cases that mix use is not enough, like our case

```
> hello_str <- "Hello!"
> hello_str <- 'Hello!'
> hello_str <- `Hello!`
Error: object 'Hello!' not found
> complex_"var"_name <- hello_str
Error: unexpected string constant in "complex_"var""
> `complex_"var"_name` <- hello_str
> print(complex_"var"_name)
Error: unexpected string constant in "print(complex_"var""
> print(`complex_"var"_name`)
[1] "Hello!"
> `complex_"var"_name` <- "i am 6'0""
+
+
> `complex_"var"_name` <- "i am 6'0\""
> `complex_"var"_name` <- 'i am 6'0"'
Error: unexpected numeric constant in "`complex_"var"_name` <- 'i am 6'0"
> `complex_"var"_name` <- 'i am 6\'0"'
> `complex_"var"_name` <- 'i am 72"'
> `complex_"var"_name` <- "i am 6'0"
```

Will stuck here

In this lecture

- String processing with `stringr`
- **Regular Expression**
- Parsing date and time

Regular expressions

A regular expression (regex) is **a way to describe specific patterns of characters** of text.

- Regex is used to determine if a given string **matches** a specific pattern;
- When matched, it can be used to **extract** the components;
- The patterns supplied to the `stringr` functions can be a regex rather than a standard string.

Online tutorial:
https://www.regular-expressions.info/tutorial.html
https://r4ds.had.co.nz/strings.html#matching-patterns-with-regular-expressions
Cheat sheet: https://github.com/rstudio/cheatsheets/blob/main/regex.pdf

Strings are regex

```
> pattern <- ","
> mutate(murders_raw, contain_comma = str_detect(total, pattern)) |>
+ select(total, contain_comma)
# A tibble: 51 × 2
   total contain_comma
   <chr> <lgl>
 1 348   FALSE
 2 59    FALSE
 3 309   FALSE
 4 181   FALSE
 5 1,861 TRUE
 6 176   FALSE
 7 117   FALSE
 8 63    FALSE
 9 162   FALSE
10 1,041 TRUE
# … with 41 more rows
# ℹ Use `print(n = ...)` to see more rows
```

Technically any string is a regex.

Searching for a single comma can also be a minimal example of searching a regex.

```
> str_subset(reported_heights$height, "cm")
[1] "165cm"  "170 cm"
```

Searching for the pattern “cm” helps us extract the problematic entries that are in centimeters

Special character

Let’s consider a slightly more complicated example. Which of the following strings contain the pattern “cm” or “inches”?

```
> yes <- c("180 cm", "70 inches")
> no <- c("180", "70''")
> s <- c(yes, no)
> str_detect(s, "cm") | str_detect(s, "inches")
[1]  TRUE  TRUE FALSE FALSE
```

However, we don’t need to do this. The main feature that distinguishes the regex language from plain strings is that we can use special characters. These are characters with a meaning. We start by introducing | which means or.

```
> str_detect(s, "cm|inches")
[1]  TRUE  TRUE FALSE FALSE
```

Special character

```
> yes <- c("5", "6", "5'10", "5 feet", "4'11")
> no <- c("", ".", "Five", "six")
> s <- c(yes, no)
> pattern <- "\\d"
> str_detect(s, pattern)
[1]  TRUE  TRUE  TRUE  TRUE  TRUE FALSE FALSE FALSE FALSE
```

`\\d` means any numeric character: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.

- The second backslash here is used to distinguish it from the character `d`. (escape of regex)
- The first backslash is used to escape the regex backslash in R. (escape of R)

```
> str_view_all(s, pattern)
[1] | <5>
[2] | <6>
[3] | <5>'<1><0>
[4] | <5> feet
[5] | <4>'<1><1>
[6] |
[7] | .
[8] | Five
[9] | six
```

We take this opportunity to introduce the `str_view_all` function, which is helpful for troubleshooting as it shows us the matches for each string.

More special characters

There are many other special characters, use your cheatsheet.

- `\\D` stands for everything this is not `\\d`
- `\\w` stands for word character and it matches any letter, number, or underscore. It is equivalent to `[a-zA-Z0-9_]`
- `\\W` stands for everything that is not `\\w`
- `\\s` stands for space, tab, vertical tab, newline, form feed, carriage return
- `\\S` stands for everything that is not `\\s`
- …

Online tutorial:
https://www.regular-expressions.info/tutorial.html
https://r4ds.had.co.nz/strings.html#matching-patterns-with-regular-expressions
Cheat sheet: https://github.com/rstudio/cheatsheets/blob/main/regex.pdf

Character class

```
> str_view_all(s, "[56]")
[1] | <5>
[2] | <6>
[3] | <5>'10
[4] | <5> feet
[5] | 4'11
[6] |
[7] | .
[8] | Five
[9] | six
```

Character class is used to match a single character that might have multiple candidates:

- `[56]` means a character that is either 5 or 6

```
> yes <- as.character(4:7)
> no <- as.character(1:3)
> s <- c(yes, no)
> str_detect(s, "[4-7]")
[1]  TRUE  TRUE  TRUE  TRUE FALSE FALSE FALSE
```

Suppose we want to match values between 4 and 7. An easier way is to use range:

- `[4-7]` is equivalent to `[4567]`
- `[0-9]` is equivalent to `\\d`

Everything is a character in regex

- `[1-20]` does not mean 1 through 20, it means the **characters 1 through 2 OR the character 0**.
- `[a-zA-Z0-9_]` is ?

Anchor

```
> pattern <- "^\\d$"
> yes <- c("1", "5", "9")
> no <- c("12", "123", " 1", "a4", "b")
> s <- c(yes, no)
> str_view_all(s, pattern)
[1] | <1>
[2] | <5>
[3] | <9>
[4] | 12
[5] | 123
[6] |  1
[7] | a4
[8] | b
```

What if we only want strings that contain ONE numeric character?

Use anchor, which let us define patterns that must start or end at a specific place.

The two most common anchors are `^` and `$` which represent the beginning and end of a string.

- What does `^\\d$` mean?
- How about `^\\d.\\d$`?
- How about `^\\^\\$$`?

Negation operator

```
> pattern <- "[^a-zA-Z]\\d"
> yes <- c(".3", "+2", "-0","*4")
> no <- c("A3", "B2", "C0", "E4")
> str_detect(yes, pattern)
[1] TRUE TRUE TRUE TRUE
> str_detect(no, pattern)
[1] FALSE FALSE FALSE FALSE
```

Another usage of the caret symbol ^ is the negation operator in character class.

As special characters:

- ^ at the beginning of the pattern
- ^ immediately after left bracket

```
> pattern <- "^[a-z^A-Z]$"
> str_view_all(
+     c("d", "E", "^"),
+     pattern)
[1] | <d>
[2] | <E>
[3] | <^>
> str_view_all(
+     c("d", "E", "^"),
+     "^[\\^]$")
[1] | d
[2] | E
[3] | <^>
```

If ^ does not appear immediately after left bracket, it is simply treated as an option for the character class.

To avoid confusion, use \\^ if you want to match a caret symbol in the actual string

Quantifier

```
> pattern <- "^\\d{1,2}$"
> yes <- c("1", "5", "9", "12")
> no <- c("123", "a4", "b")
> str_view_all(c(yes, no), pattern)
[1] | <1>
[2] | <5>
[3] | <9>
[4] | <12>
[5] | 123
[6] | a4
[7] | b
```

How can we match strings that contain a number that is in the range of [0, 99]?

Use quantifier, which are curly brackets containing the number of times the previous entry can be repeated.

- `^\\d{1,2}$` can match texts with either 1 or 2 numeric digits
- `^\\d{1,6}$` can match texts with 1-6 numeric digits, inclusive

Arbitrary placement of spaces

```
> paste(problems, collapse="  |  ")
[1] "6  |  5' 4\"  |  5.3  |  165cm  |  511  |  6  |  2  |  5'7  |  >9000  |  5'7
\"  |  5'3\"  |  5 feet and 8.11 inches  |  5.25  |  5'11  |  5.5  |  11111  |
5'9''  |  6  |  6.5  |  150  |  5'10''  |  103.2  |  5.8  |  19  |  5  |  5.6  |
175  |  177  |  300  |  5,3  |  6'  |  6  |  5.9  |  6,8  |  5' 10  |  5.5  |  17
8  |  163  |  6.2  |  175  |  Five foot eight inches  |  6.2  |  5.8  |  5.1  |
178  |  165  |  5.11  |  5'5\"  |  165  |  180  |  5'2\"  |  5.75  |  169  |  5,4
|  7  |  5.4  |  157  |  6.1  |  169  |  5'3  |  5.6  |  214  |  183  |  5.6  |
6  |  162  |  178  |  180  |  5'10''  |  170  |  5'3''  |  178  |  0.7  |  190  |
5.4  |  184  |  5'7''  |  5.9  |  5'12  |  5.6  |  5.6  |  184  |  6  |  167  |
2'33  |  5'11  |  5'3\"  |  5.5  |  5.2  |  180  |  5.5  |  5.5  |  6.5  |  5,8
|  180  |  183  |  170  |  5'6''  |  172  |  612  |  5.11  |  168  |  5'4  |  1,7
0  |  172  |  87  |  5.5  |  176  |  5'7.5''  |  5'7.5''  |  111  |  5'2\"  |  17
3  |  174  |  176  |  175  |  5' 7.78\"  |  6.7  |  12  |  6  |  5.1  |  5.6  |
5.5  |  yyy  |  5.2  |  5'5  |  5'8  |  5'6  |  5 feet 7inches  |  89  |  5.6  |
5.7  |  183  |  172  |  34  |  25  |  6  |  5.9  |  168  |  6.5  |  170  |  175
|  6  |  22  |  5.11  |  684  |  6  |  1  |  1  |  6*12  |  5 .11  |  87  |  162
|  165  |  184  |  6  |  173  |  1.6  |  172  |  170  |  5.7  |  5.5  |  174  |
170  |  160  |  120  |  120  |  23  |  192  |  5 11  |  167  |  150  |  1.7  |  1
74  |  5.8  |  6  |  5'4  |  5'8\"  |  5'5  |  5.8  |  5.1  |  5.11  |  5.7  |
5'7  |  5'6  |  5'11\"  |  5'7\"  |  5'7  |  172  |  5'8  |  180  |  5' 11\"  |
5  |  180  |  180  |  6'1\"  |  5.9  |  5.2  |  5.5  |  69\"  |  5' 7\"  |  5'1
0''  |  5.51  |  5'10  |  5'10  |  5ft 9 inches  |  5 ft 9 inches  |  5'2  |  5'1
1  |  5.8  |  5.7  |  167  |  168  |  6  |  6.1  |  5'11''  |  5.69  |  178  |  1
82  |  164  |  5'8\"  |  185  |  6  |  86  |  5.7  |  708,661  |  5.25  |  5.5  |
5 feet 6 inches  |  5'10''  |  172  |  6  |  5'8  |  160  |  6'3\"  |  649,606  |
10000  |  5.1  |  152  |  1  |  180  |  728,346  |  175  |  158  |  173  |  164
|  6 04  |  169  |  0  |  185  |  168  |  5'9  |  169  |  5'5''  |  174  |  6.3
|  179  |  5'7\"  |  5.5  |  6  |  6  |  170  |  6  |  172  |  158  |  100  |  15
9  |  190  |  5.7  |  170  |  158  |  6'4\"  |  180  |  5.57  |  5'4  |  210  |
88  |  6  |  162  |  170 cm  |  5.7  |  170  |  157  |  186  |  170  |  7,283,465
```

In noisy data, space can appear almost anywhere.

Can we just remove all spaces?

How about this one: “5 11”

```
> pattern_2 <- "^[4-7]'\\s\\d{1,2}\"$"
> str_subset(problems, pattern_2)
[1] "5' 4\""   "5' 11\"" "5' 7\""
```

It’s usually more recommended to account for spaces in your regex:

- Adding a `\\s` allows a space
- Then how about patterns with no space?

Quantifier continued

```
> yes <- c("AB", "A1B", "A11B", "A111B", "A1111B")
> data.frame(string = c("AB", "A1B", "A11B", "A111B", "A1111B"),
+            none_or_more = str_detect(yes, "A1*B"),
+            nore_or_once = str_detect(yes, "A1?B"),
+            once_or_more = str_detect(yes, "A1+B"))
  string none_or_more nore_or_once once_or_more
1     AB          TRUE          TRUE        FALSE
2    A1B          TRUE          TRUE         TRUE
3   A11B          TRUE         FALSE         TRUE
4  A111B          TRUE         FALSE         TRUE
5 A1111B          TRUE         FALSE         TRUE
```

`*`: match 0,1,2,3,… instances

`?`: match 0 or 1 instance

`+`: match 1,2,3,… instances

Use * with caution!

Wildcard

```
> yes <- c("1", "5", "9", "12")
> no <- c("123", "a4", "b")
> pattern <- "^.$"
> str_view_all(c(yes, no), pattern)
[1] | <1>
[2] | <5>
[3] | <9>
[4] | 12
[5] | 123
[6] | a4
[7] | <b>
> pattern <- "^..$"
> str_view_all(c(yes, no), pattern)
[1] | 1
[2] | 5
[3] | 9
[4] | <12>
[5] | 123
[6] | <a4>
[7] | b
> pattern <- "^...$"
> str_view_all(c(yes, no), pattern)
[1] | 1
[2] | 5
[3] | 9
[4] | 12
[5] | <123>
[6] | a4
[7] | b
```

. means matching any single character

. . means matching any two characters

. . . …

. * means matching whatever (avoid)

```
> pattern <- "^.*$"
> str_view_all(c(yes, no), pattern)
[1] | <1>
[2] | <5>
[3] | <9>
[4] | <12>
[5] | <123>
[6] | <a4>
[7] | <b>
```

Regex learned so far

- Explicit match: `1`
- Character class: `[123abc]`
- Wildcard: `.` `\\d` `\\w`
- Space: `\\s`
- Anchors: `^$`
- Negator: `[^1]`
- Quantifier: `{1,3}` `*` `?` `+`

Combine the patterns learned so far

```
> pattern <- "^[4-7]\\s*'\\s*\\d{1,2}\"$"
```

^ = start of the string

[4-7] = one digit, either 4,5,6 or 7

\\s* = arbitrary amount of spaces

' = feet symbol

\\d{1,2} = one or two numeric digits

\" = inches symbol, note the escape for R

$ = end of the string

Lookaround

Lookaround provides a way to ask for one or more conditions to be satisfied **without moving the search forward or matching it**.

Lookaround will not be used in following case studies. It is also not recommended in the data wrangling routine.

```
> pattern <- "(?=^\\w{8,16}$)(?=^[a-z|A-Z].*)(?=.*\\d+.*).*"
> yes <- c("Ihatepasswords1", "password1234")
> no <- c("sh0rt", "Ihaterpasswords", "7X%9,N`yrYG92b7")
> str_detect(yes, pattern)
[1] TRUE TRUE
> str_detect(no, pattern)
[1] FALSE FALSE FALSE
```

Four types of lookaround:

- lookahead `(?=)`
- lookbehind `(?<=)`
- negative lookahead `(?!)`
- negative lookbehind `(?<!)`

On the left shows an example that checks if a password satisfies the following conditions:

1) 8-16 alphanumerical characters
2) starts with a letter
3) has at least one digit

Why you should avoid lookaround

```
> pattern <- "(?=^\\w{8,16}$)(?=^[a-z|A-Z].*)(?=.*\\d+.*).*"
> yes <- c("Ihatepasswords1", "password1234")
> no <- c("sh0rt", "Ihaterpasswords", "7X%9,N`yrYG92b7")
> str_detect(yes, pattern)
[1] TRUE TRUE
> str_detect(no, pattern)
[1] FALSE FALSE FALSE
```

```
> pattern1 <- "^\\w{8,16}$"
> pattern2 <- "^[a-z|A-Z]"
> pattern3 <- ".*\\d+.*"
> str_detect(yes, pattern1) &
+ str_detect(yes, pattern2) &
+ str_detect(yes, pattern3)
[1] TRUE TRUE
> str_detect(no, pattern1) &
+ str_detect(no, pattern2) &
+ str_detect(no, pattern3)
[1] FALSE FALSE FALSE
```

- Regex with lookaround is less readable
- Lookaround + quantifier (+, *) could substantially slow you down
- Alternatives by using and/or logics almost always exist, the example above can be replaced by three individual regular expressions without lookaround.

Group ()

```
> pattern_without_groups <- "^[4-7],\\d*$"
> pattern_with_groups <-  "^([4-7]),(\\d*)$"
> yes <- c("5,9", "5,11", "6,", "6,1")
> no <- c("5'9", ",", "2,8", "6.1.1")
> s <- c(yes, no)
> str_detect(s, pattern_without_groups)
[1]  TRUE  TRUE  TRUE  TRUE FALSE FALSE FALSE FALSE
> yes <- c("5,9", "5,11", "6,", "6,1")
> no <- c("5'9", ",", "2,8", "6.1.1")
> s <- c(yes, no)
> str_detect(s, pattern_with_groups)
[1]  TRUE  TRUE  TRUE  TRUE FALSE FALSE FALSE FALSE
> str_match(s, pattern_with_groups)
     [,1]   [,2] [,3]
[1,] "5,9"  "5"  "9"
[2,] "5,11" "5"  "11"
[3,] "6,"   "6"  ""
[4,] "6,1"  "6"  "1"
[5,] NA     NA   NA
[6,] NA     NA   NA
[7,] NA     NA   NA
[8,] NA     NA   NA
> str_extract(s, pattern_with_groups)
[1] "5,9"  "5,11" "6,"   "6,1"  NA     NA     NA     NA
```

- So far, we have been using regex for matching strings, another important usage of regex is to extract the matched part(s);
- Use group `()` to indicate which part(s) you want to extract;
- Use `str_extract` to extract matches of the whole pattern;
- Use `str_match` to extract both the whole pattern and the groups defined by parentheses.

Search and replace with regex

```
> pattern <- "^[4-7]'\\d{1,2}\"$"
> sum(str_detect(problems, pattern))
[1] 14
```

```
> problems[!str_detect(problems, pattern)]
 [1] "6"                          "5' 4\""
 [3] "5.3"                        "165cm"
 [5] "511"                        "6"
 [7] "2"                          "5'7"
 [9] ">9000"                      "5 feet and 8.11 inches"
[11] "5.25"                       "5'11"
[13] "5.5"                        "11111"
[15] "5'9''"                      "6"
```

```
> str_subset(problems, "inches")
[1] "5 feet and 8.11 inches" "Five foot eight inches"
[3] "5 feet 7inches"         "5ft 9 inches"
[5] "5 ft 9 inches"          "5 feet 6 inches"
```

```
> str_subset(problems, "''")
 [1] "5'9''"   "5'10''"  "5'10''"  "5'3''"   "5'7''"   "5'6''"
 [7] "5'7.5''" "5'7.5''" "5'10''"  "5'11''"  "5'10''"  "5'5''"
```

In `reported_heights`, problematic entries also have common patterns:

- How can we detect these patterns?
- How can we unify the patterns to (feet)'(inches)?
- How can we extract the information?

Search and replace with regex

```
> problems |>
+ str_replace("feet|ft|foot", "'") |>
+ str_replace("inches|in|''|\"", "") |>
+ str_detect("^[4-7]'\\d{1,2}$") |>
+ sum()
[1] 48
```

1. Replace all characters/words that mean “foot” to a single quote;
2. Identify and remove all signs/words that mean “inches”;

=> 48 entries corrected

```
> "5 feet 6 inches" |>
+ str_replace("feet|ft|foot", "'") |>
+ str_replace("inches|in|''|\"", "")
[1] "5 ' 6 "
```

Failure mode: extra spaces;
How to account for it?

```
> problems |>
+ str_replace("feet|ft|foot", "'") |>
+ str_replace("inches|in|''|\"", "") |>
+ str_detect("^[4-7]\\s*'\\s*\\d{1,2}$") |>
+ sum()
[1] 53
```

3. allowing spaces in the regex

=> 5 more entries corrected

Search and replace using group

Next major problematic form: x.y, x,y and x y.

- We don’t want to just do a search and replace. Why? (think about 70.5)
- We should put constraint on the numbers before and after the separator;
- We can detect forms that have reasonable numbers, extract the numbers and recompose the output.

Regex allows us to reuse the i-th extracted groups by `\\i`:

```
> pattern_with_groups <-  "^([4-7]),(\\d*)$"
> yes <- c("5,9", "5,11", "6,", "6,1")
> no <- c("5'9", ",", "2,8", "6.1.1")
> s <- c(yes, no)
> str_replace(s, pattern_with_groups, "\\1'\\2")
[1] "5'9"   "5'11"  "6'"    "6'1"   "5'9"   ","     "2,8"   "6.1.1"
```

Search and replace using group

```
> pattern_with_groups <-"^([4-7])[,\\.\\s+](\\d*)$"
> str_subset(problems, pattern_with_groups)
 [1] "5.3"  "5.25" "5.5"  "6.5"  "5.8"  "5.6"  "5,3"  "5.9"  "6,8"
[10] "5.5"  "6.2"  "6.2"  "5.8"  "5.1"  "5.11" "5.75" "5,4"  "5.4"
[19] "6.1"  "5.6"  "5.6"  "5.4"  "5.9"  "5.6"  "5.6"  "5.5"  "5.2"
[28] "5.5"  "5.5"  "6.5"  "5,8"  "5.11" "5.5"  "6.7"  "5.1"  "5.6"
[37] "5.5"  "5.2"  "5.6"  "5.7"  "5.9"  "6.5"  "5.11" "5.7"  "5.5"
[46] "5 11" "5.8"  "5.8"  "5.1"  "5.11" "5.7"  "5.9"  "5.2"  "5.5"
[55] "5.51" "5.8"  "5.7"  "6.1"  "5.69" "5.7"  "5.25" "5.5"  "5.1"
[64] "6 04" "6.3"  "5.5"  "5.7"  "5.57" "5.7"
> str_subset(problems, pattern_with_groups) |>
+ str_replace(pattern_with_groups, "\\1'\\2")
 [1] "5'3"  "5'25" "5'5"  "6'5"  "5'8"  "5'6"  "5'3"  "5'9"  "6'8"
[10] "5'5"  "6'2"  "6'2"  "5'8"  "5'1"  "5'11" "5'75" "5'4"  "5'4"
[19] "6'1"  "5'6"  "5'6"  "5'4"  "5'9"  "5'6"  "5'6"  "5'5"  "5'2"
[28] "5'5"  "5'5"  "6'5"  "5'8"  "5'11" "5'5"  "6'7"  "5'1"  "5'6"
[37] "5'5"  "5'2"  "5'6"  "5'7"  "5'9"  "6'5"  "5'11" "5'7"  "5'5"
[46] "5'11" "5'8"  "5'8"  "5'1"  "5'11" "5'7"  "5'9"  "5'2"  "5'5"
[55] "5'51" "5'8"  "5'7"  "6'1"  "5'69" "5'7"  "5'25" "5'5"  "5'1"
[64] "6'04" "6'3"  "5'5"  "5'7"  "5'57" "5'7"
```

Testing and improving

```
> not_inches_or_cm <- function(x, smallest = 50, tallest = 84){
+     inches <- suppressWarnings(as.numeric(x))
+     ind <- !is.na(inches) &
+         ((inches >= smallest & inches <= tallest) |
+              (inches/2.54 >= smallest & inches/2.54 <= tallest))
+     !ind
+ }
> problems <- reported_heights |>
+ filter(not_inches_or_cm(height)) |>
+ pull(height)
> length(problems)
[1] 200
```

```
> converted <- problems |>
+ str_replace("feet|foot|ft", "'") |>
+ str_replace("inches|in|''|\"", "") |>
+ str_replace("^([4-7])[,\\.\\s+](\\d*)$", "\\1'\\2")
>
> pattern <- "^[4-7]\\s*'\\s*\\d{1,2}$"
> index <- str_detect(converted, pattern)
> mean(index)
[1] 0.61
```

So far, we have accounted for:

- Different feet/inch signs;
- Spaces in the string;
- Different separators;

⇒61% of the problematic entries are now corrected.

Takeaway: trial and error is a common approach to iteratively refine your regex pattern

Testing and improving

```
> converted[!index]
 [1] "6"             "165cm"         "511"           "6"
 [5] "2"             ">9000"         "5 ' and 8.11 " "11111"
 [9] "6"             "103.2"         "19"            "5"
[13] "300"           "6'"            "6"             "Five ' eight "
[17] "7"             "214"           "6"             "0.7"
[21] "6"             "2'33"          "612"           "1,70"
[25] "87"            "5'7.5"         "5'7.5"         "111"
[29] "5' 7.78"       "12"            "6"             "yyy"
[33] "89"            "34"            "25"            "6"
[37] "6"             "22"            "684"           "6"
[41] "1"             "1"             "6*12"          "5 .11"
[45] "87"            "6"             "1.6"           "120"
[49] "120"           "23"            "1.7"           "6"
[53] "5"             "69"            "5' 9 "         "5 ' 9 "
[57] "6"             "6"             "86"            "708,661"
[61] "5 ' 6 "        "6"             "649,606"       "10000"
[65] "1"             "728,346"       "0"             "6"
[69] "6"             "6"             "100"           "88"
[73] "6"             "170 cm"        "7,283,465"     "5"
[77] "5"             "34"
```

What's left?

Major patterns:

1. Exactly 5 or 6 feet, used a single number or a single number + single quote: 5, 5'
2. Combination of quote and decimal points: 5'7.5''
3. Spaces elsewhere: "5'9 "
4. Number as word: Five ' eight
5. Centimeter measurements: 170 cm

Testing and improving

```
> converted <- problems |>
+   str_replace("feet|foot|ft", "'") |> # feet symbols
+   str_replace("inches|in|''|\"", "") |>  # remove inches symbols
+   str_replace("^([4-7])\\s*[,\\.\\s+]\\s*(\\d*)$", "\\1'\\2") |>
+   str_replace("^([56])'?$", "\\1'0")
>
> pattern <- "^[4-7]\\s*'\\s*\\d{1,2}$"
> index <- str_detect(converted, pattern)
> mean(index)
[1] 0.735
```

What's left?

Major patterns:

1. Exactly 5 or 6 feet, used a single number or a single number + single quote: 5, 5'
2. Combination of quote and decimal points: 5'7.5''
3. Spaces elsewhere: "5'9 "
4. Number as word: Five ' eight
5. Centimeter measurements: 170 cm

Testing and improving

```
> converted <- problems |>
+   str_replace("feet|foot|ft", "'") |> # feet symbols
+   str_replace("inches|in|''|\"", "") |>  # remove inches symbols
+   str_replace("^([4-7])\\s*[,\\.\\s+]\\s*(\\d*)$", "\\1'\\2") |>
+   str_replace("^([56])'?$", "\\1'0") |>
+   str_trim()
>
> pattern <- "^[4-7]\\s*'\\s*\\d{1,2}$"
> index <- str_detect(converted, pattern)
> mean(index)
[1] 0.75
```

What's left?

Major patterns:

1. Exactly 5 or 6 feet, used a single number or a single number + single quote: 5, 5'
2. Combination of quote and decimal points: 5'7.5''
3. Spaces elsewhere: "5'9 "
4. Number as word: Five ' eight
5. Centimeter measurements: 170 cm

Testing and improving

```
> words_to_numbers <- function(s){
+   s <- str_to_lower(s)
+   for(i in 0:11)
+     s <- str_replace_all(s, words(i), as.character(i))
+   s
+ }
>
> converted <- problems |>
+   words_to_numbers() |>
+   str_replace("feet|foot|ft", "'") |> # feet symbols
+   str_replace("inches|in|''|\"", "") |>  # remove inches symbols
+   str_replace("^([4-7])\\s*[,\\.\\s+]\\s*(\\d*)$", "\\1'\\2") |>
+   str_replace("^([56])'?$", "\\1'0") |>
+   str_trim()
>
> pattern <- "^[4-7]\\s*'\\s*\\d{1,2}$"
> index <- str_detect(converted, pattern)
> mean(index)
[1] 0.755
```

What's left?

Major patterns:

1. Exactly 5 or 6 feet, used a single number or a single number + single quote: 5, 5'
2. Combination of quote and decimal points: 5'7.5''
3. Spaces elsewhere: "5'9 "
4. Number as word: Five ' eight
5. Centimeter measurements: 170 cm

Testing and improving

```
> converted <- problems |>
+   words_to_numbers() |>
+   str_replace("feet|foot|ft", "'") |> # feet symbols
+   str_replace("inches|in|''|\"", "") |>  # remove inches symbols
+   str_replace("^([4-7])\\s*[,\\.\\s+]\\s*(\\d*)$", "\\1'\\2") |>
+   str_replace("^([56])'?$", "\\1'0") |>
+   str_replace("^([12]\\d{2})\\s*cm$", "\\1") |>
+   str_trim()
>
> pattern <- "^[4-7]\\s*'\\s*\\d{1,2}$"
> index <- str_detect(converted, pattern)
> mean(index)
[1] 0.755
```

What's left?

Major patterns:

1. Exactly 5 or 6 feet, used a single number or a single number + single quote: 5, 5'
2. Combination of quote and decimal points: 5'7.5''
3. Spaces elsewhere: "5'9 "
4. Number as word: Five ' eight
5. Centimeter measurements: 170 cm

Testing and improving

```
> converted <- problems |>
+   words_to_numbers() |>
+   str_replace("feet|foot|ft", "'") |> # feet symbols
+   str_replace("inches|in|''|\"", "") |>  # remove inches symbols
+   str_replace("^([4-7])\\s*[,\\.\\s+]\\s*(\\d*)$", "\\1'\\2") |>
+   str_replace("^([56])'?$", "\\1'0") |>
+   str_replace("^([12]\\d{2})\\s*cm$", "\\1") |>
+   str_trim()
>
> pattern <- "^[4-7]\\s*'\\s*(\\d+\\.?\\d*)$"
> index <- str_detect(converted, pattern)
> mean(index)
[1] 0.77
```

What's left?

Major patterns:

1. Exactly 5 or 6 feet, used a single number or a single number + single quote: 5, 5'
2. Combination of quote and decimal points: 5'7.5''
3. Spaces elsewhere: "5'9 "
4. Number as word: Five ' eight
5. Centimeter measurements: 170 cm

Testing and improving

```
> words_to_numbers <- function(s){
+   s <- str_to_lower(s)
+   for(i in 0:11)
+     s <- str_replace_all(s, words(i), as.character(i))
+   s
+ }
>
> convert_format <- function(s){
+     s |>
+     str_replace("feet|foot|ft", "'") |> # feet symbols
+     str_replace("inches|in|''|\"", "") |>  # remove inches symbols
+     str_replace("^([4-7])\\s*[,\\.\\s+]\\s*(\\d*)$", "\\1'\\2") |>
+     str_replace("^([567])'?$", "\\1'0") |>
+     str_replace("^([12]\\d{2})\\s*cm$", "\\1") |>
+     str_trim()
+ }
>
> converted <- problems |> words_to_numbers() |> convert_format()
> remaining <- converted[not_inches_or_cm(converted)]
>
> pattern <- "^[4-7]\\s*'\\s*(\\d+\\.?\\d*)$"
> index <- str_detect(remaining, pattern)
> remaining[!index]
 [1] "511"          "2"            ">9000"        "5 ' and 8.11" "11111"
 [6] "103.2"        "19"           "300"          "214"          "0.7"
[11] "2'33"         "612"          "1,70"         "87"           "111"
[16] "12"           "yyy"          "89"           "34"           "25"
[21] "22"           "684"          "1"            "1"            "6*12"
[26] "87"           "1.6"          "120"          "120"          "23"
[31] "1.7"          "86"           "708,661"      "649,606"      "10000"
[36] "1"            "728,346"      "0"            "100"          "88"
[41] "7,283,465"    "34"
```

What's left?

Last stretch: parse feet and inches

```
> remaining[index]
  [1] "6'0"   "5' 4"  "5'3"   "6'0"   "5'7"   "5'7"   "5'3"
  [8] "5'25"  "5'11"  "5'5"   "5'9"   "6'0"   "6'5"   "5'10"
 [15] "5'8"   "5'0"   "5'6"   "5'3"   "6'0"   "6'0"   "5'9"
 [22] "6'8"   "5' 10" "5'5"   "6'2"   "5 ' 8" "6'2"   "5'8"
 [29] "5'1"   "5'11"  "5'5"   "5'2"   "5'75"  "5'4"   "5'4"
 [36] "6'1"   "5'3"   "5'6"   "5'6"   "6'0"   "5'10"  "5'3"
 [43] "5'4"   "5'7"   "5'9"   "5'12"  "5'6"   "5'6"   "6'0"
 [50] "5'11"  "5'3"   "5'5"   "5'2"   "5'5"   "5'5"   "6'5"
 [57] "5'8"   "5'6"   "5'11"  "5'4"   "5'5"   "5'2"   "6'7"
 [64] "6'0"   "5'1"   "5'6"   "5'5"   "5'2"   "5'5"   "5'8"
 [71] "5'6"   "5 ' 7" "5'6"   "5'7"   "6'0"   "5'9"   "6'5"
 [78] "6'0"   "5'11"  "6'0"   "5'11"  "6'0"   "5'7"   "5'5"
 [85] "5'11"  "5'8"   "6'0"   "5'4"   "5'8"   "5'5"   "5'8"
 [92] "5'1"   "5'11"  "5'7"   "5'7"   "5'6"   "5'11"  "5'7"
 [99] "5'7"   "5'8"   "5' 11" "5'0"   "6'1"   "5'9"   "5'2"
[106] "5'5"   "5' 7"  "5'10"  "5'51"  "5'10"  "5'10"  "5' 9"
[113] "5 ' 9" "5'2"   "5'11"  "5'8"   "5'7"   "6'0"   "6'1"
[120] "5'11"  "5'69"  "5'8"   "6'0"   "5'7"   "5'25"  "5'5"
[127] "5 ' 6" "5'10"  "6'0"   "5'8"   "6'3"   "5'1"   "6'04"
[134] "5'9"   "5'5"   "6'3"   "5'7"   "5'5"   "6'0"   "6'0"
[141] "6'0"   "5'7"   "6'4"   "5'57"  "5'4"   "6'0"   "5'7"
[148] "5'0"   "5'0"   "5'6"   "5'6"
```

```
> s <- c("5'10", "6'1")
> tab <- data.frame(x = s)
> tab |>
+     separate(x, c("feet", "inches"), sep="'") |>
+     mutate(feet = as.numeric(feet)) |>
+     mutate(inches = as.numeric(inches)) |>
+     mutate(height = feet * 12 + inches)
  feet inches height
1    5     10     70
2    6      1     73
```

We can use `separate` now

Last stretch: parse feet and inches

```
> s <- c("5'10", "6'1")
> tab <- data.frame(x = s)
> tab |>
+   extract(x, c("feet", "inches"), regex="(\\d)'(\\d{1,2})") |>
+   mutate(feet = as.numeric(feet)) |>
+   mutate(inches = as.numeric(inches)) |>
+   mutate(height = feet * 12 + inches)
  feet inches height
1    5     10     70
2    6      1     73
```

Or even better, we can use regex through `extract`

Altogether

```
pattern <- "^([4-7])\\s*'\\s*(\\d+\\.?\\d*)$"

smallest <- 50
tallest <- 84
new_heights <- reported_heights |>
  mutate(original = height,
         height = words_to_numbers(height) |> convert_format()) |>
  extract(height, c("feet", "inches"), regex = pattern, remove = FALSE) |>
  mutate(height = as.numeric(height, ),
         feet = as.numeric(feet),
         inches = as.numeric(inches)) |>
  # Transform feet'inches to inches
  mutate(guess = 12 * feet + inches) |>
  # Transform measurements in centimeter and meter to inches
  # Also filter for outliers
  mutate(height = case_when(
      is.na(height) ~ as.numeric(NA),
      between(height, smallest, tallest) ~ height,  #inches
      between(height/2.54, smallest, tallest) ~ height/2.54, #cm
      between(height*100/2.54, smallest, tallest) ~ height*100/2.54, #meters
      TRUE ~ as.numeric(NA))) |>
  # Remove erroneous and na entries
  mutate(height = ifelse(is.na(height) &
                         inches < 12 &
                         between(guess, smallest, tallest),
                         guess, height)) |>
  select(-guess)
```

```
> new_heights
             time_stamp    sex   height feet inches original
1   2014-09-02 13:40:36   Male 75.00000   NA     NA       75
2   2014-09-02 13:46:59   Male 70.00000   NA     NA       70
3   2014-09-02 13:59:20   Male 68.00000   NA     NA       68
4   2014-09-02 14:51:53   Male 74.00000   NA     NA       74
5   2014-09-02 15:16:15   Male 61.00000   NA     NA       61
6   2014-09-02 15:16:16 Female 65.00000   NA     NA       65
7   2014-09-02 15:16:19 Female 66.00000   NA     NA       66
8   2014-09-02 15:16:21 Female 62.00000   NA     NA       62
9   2014-09-02 15:16:21 Female 66.00000   NA     NA       66
10  2014-09-02 15:16:22   Male 67.00000   NA     NA       67
11  2014-09-02 15:16:22   Male 72.00000   NA     NA       72
12  2014-09-02 15:16:23   Male 72.00000    6      0        6
13  2014-09-02 15:16:23   Male 69.00000   NA     NA       69
14  2014-09-02 15:16:26   Male 68.00000   NA     NA       68
15  2014-09-02 15:16:26   Male 69.00000   NA     NA       69
16  2014-09-02 15:16:26   Male 66.00000   NA     NA       66
17  2014-09-02 15:16:27   Male 75.00000   NA     NA       75
18  2014-09-02 15:16:27 Female 64.00000   NA     NA       64
19  2014-09-02 15:16:27 Female 60.00000   NA     NA       60
20  2014-09-02 15:16:28   Male 67.00000   NA     NA       67
21  2014-09-02 15:16:28   Male 66.00000   NA     NA       66
22  2014-09-02 15:16:28   Male 64.00000    5      4    5' 4"
23  2014-09-02 15:16:28   Male 70.00000   NA     NA       70
24  2014-09-02 15:16:29   Male 73.00000   NA     NA       73
25  2014-09-02 15:16:29   Male 72.00000   NA     NA       72
26  2014-09-02 15:16:29   Male 69.00000   NA     NA       69
27  2014-09-02 15:16:29   Male 69.00000   NA     NA       69
```

Try Regex yourself

Finish the exercises at: https://regexone.com/

In this lecture

- String processing with `stringr`
- Regular Expression
- Parsing date and time

Parsing date and time

Another common atomic data type: dates/times.

- Though dates can be written as strings, the internal order is lost;
- Computer languages usually use January 1, 1970 as a reference day, referred to as the epoch. Other dates can be transformed into integer accordingly: January 2, 1970 is day 1, December 31, 1969 is day -1, and November 2, 2017, is day 17204

R defines a specific data type for dates and times:

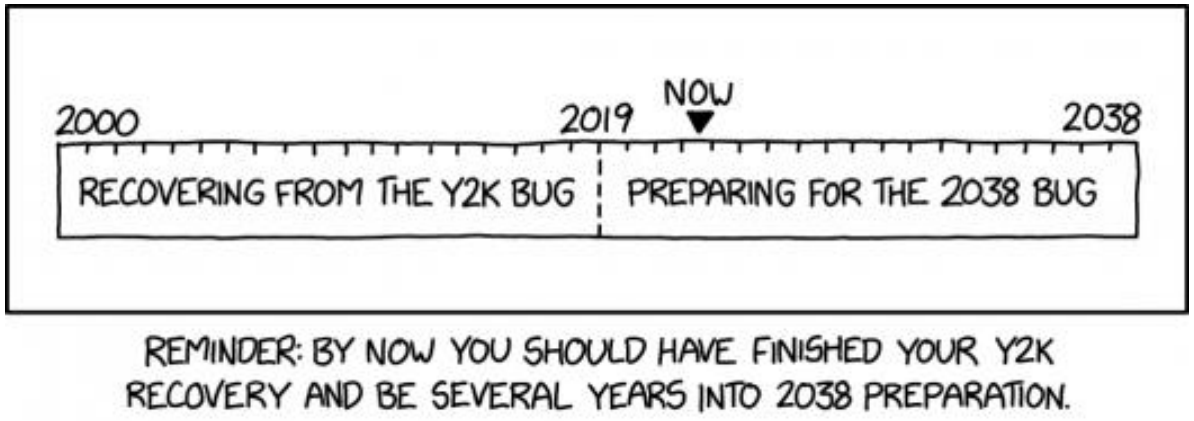
```
> library(tidyverse)
> library(dslabs)
> data("polls_us_election_2016")
> polls_us_election_2016$startdate |> head()
[1] "2016-11-03" "2016-11-01" "2016-11-02" "2016-11-04"
[5] "2016-11-03" "2016-11-03"
> class(polls_us_election_2016$startdate)
[1] "Date"
```

The epochalypse

The Unix epoch - number of seconds elapsed since *00:00:00 UTC on 1 January 1970*

This is stored as a signed 32-bit integer, thus maxed at $2^{31} - 1$

03:14:07 UTC on 19 January 2038 will be the $(2^{31} - 1)$ second since the Unix epoch



Parsing date and time

Look at what happens when we convert them to numbers:

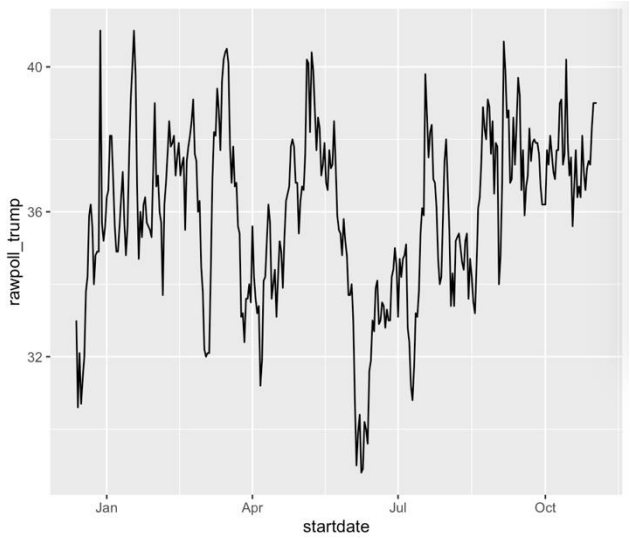
```
> as.numeric(polls_us_election_2016$startdate) |> head()
[1] 17108 17106 17107 17109 17108 17108
```

The `as.Date` function can convert a character into a date:

```
> as.Date("1970-01-01") |> as.numeric()
[1] 0
```

Plotting functions are aware of the date format

```
> polls_us_election_2016 |> filter(pollster == "Ipsos" & state =="U.S.") |>
+ ggplot(aes(startdate, rawpoll_trump)) +
+ geom_line()
```



The `lubridate` package

```
> library(lubridate)
> set.seed(42)
> dates <- sample(polls_us_election_2016$startdate, 10) |> sort()
> dates
 [1] "2016-02-16" "2016-02-22" "2016-05-23" "2016-09-06" "2016-09-16" "2016-09-27" "2016-10-18"
 [8] "2016-10-25" "2016-11-01" "2016-11-01"
> data.frame(date = dates, month = month(dates), day = day(dates), year = year(dates))
         date month day year
1  2016-02-16     2  16 2016
2  2016-02-22     2  22 2016
3  2016-05-23     5  23 2016
4  2016-09-06     9   6 2016
5  2016-09-16     9  16 2016
6  2016-09-27     9  27 2016
7  2016-10-18    10  18 2016
8  2016-10-25    10  25 2016
9  2016-11-01    11   1 2016
10 2016-11-01    11   1 2016
```

`tidyverse` includes functionality for dealing with dates through the `lubridate` package

```
> month(dates, label = TRUE)
 [1] Feb Feb May Sep Sep Sep Oct Oct Nov Nov
Levels: Jan < Feb < Mar < Apr < May < Jun < Jul < Aug < Sep < Oct < Nov < Dec
```

We can also extract the month labels

The `lubridate` package

```
> x <- c(20090101, "2009-01-02", "2009 01 03", "2009-1-4",
+        "2009-1, 5", "Created on 2009 1 6", "200901 !!! 07")
> ymd(x)
[1] "2009-01-01" "2009-01-02" "2009-01-03" "2009-01-04" "2009-01-05"
[6] "2009-01-06" "2009-01-07"
```

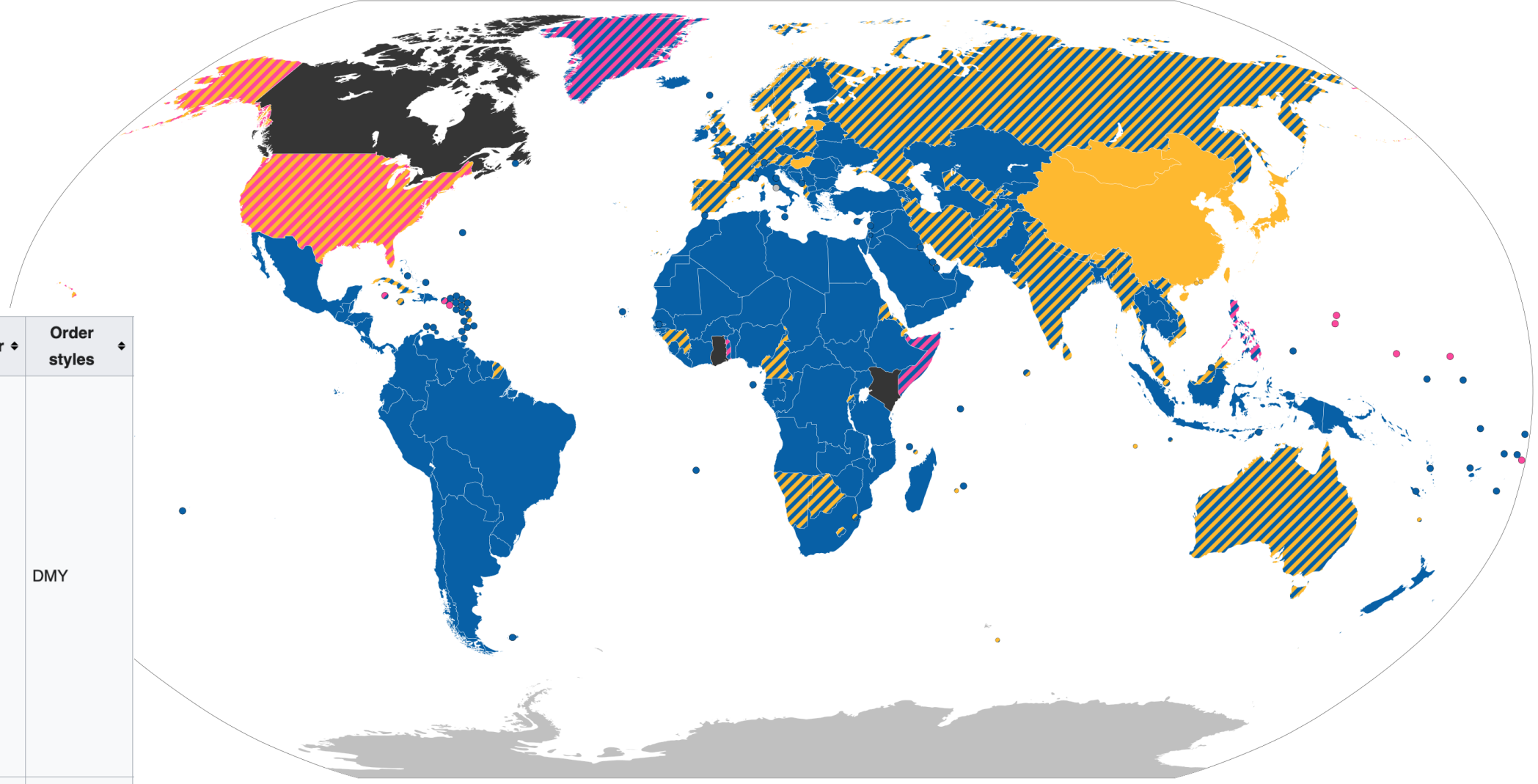
Another useful set of functions: `dmy`, `ymd`, `mdy` for parsing strings into dates.

Why not automatically parse?

```
> x <- "09/01/02"
> dmy(x)
[1] "2002-01-09"
> mdy(x)
[1] "2002-09-01"
```

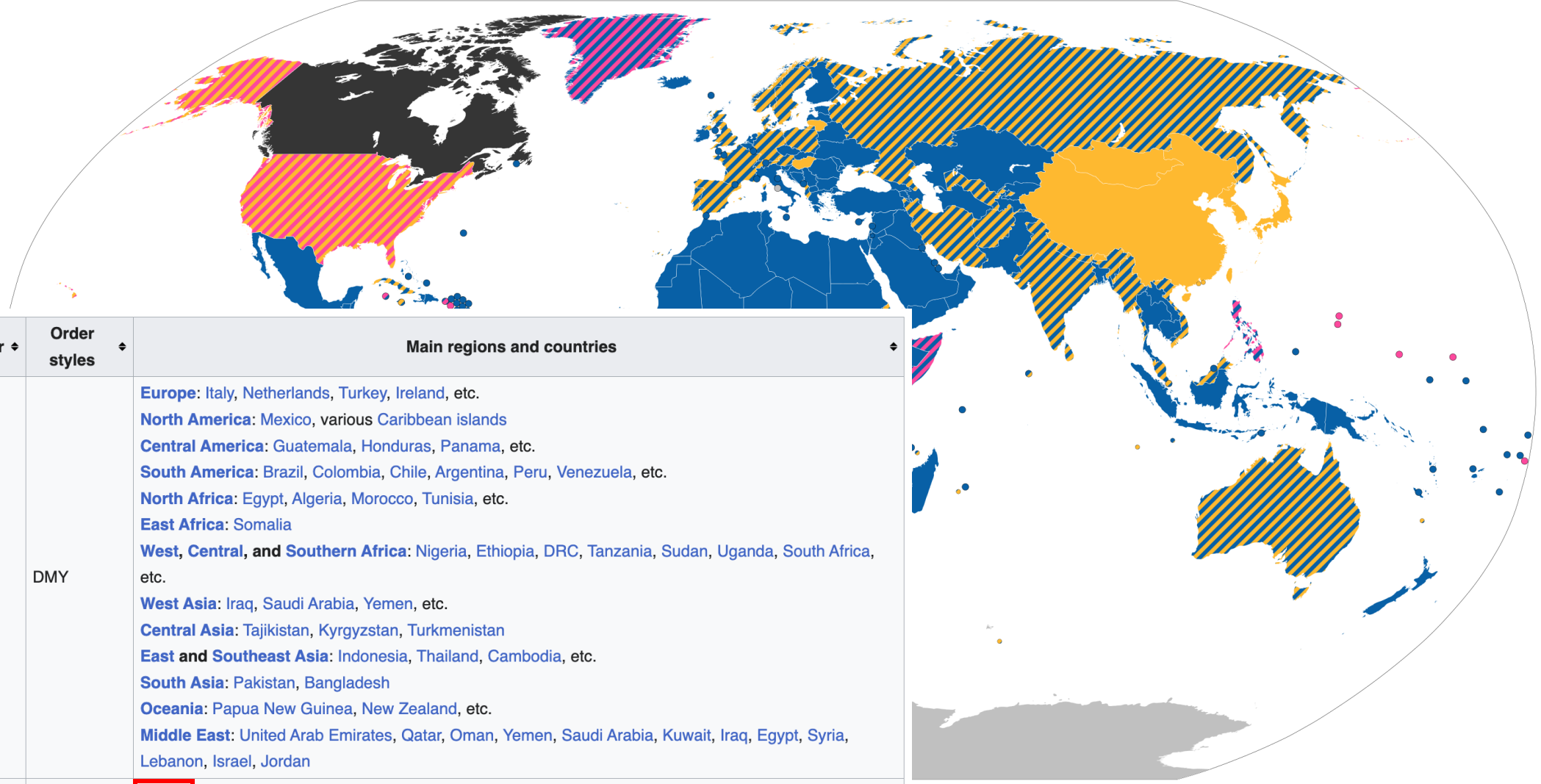
To deal with dd mm yyyy, use `dmy`

To deal with mm dd yyyy, use `mdy`



Colour	Order styles
	DMY
	YMD
	MDY
	DMY, YMD
	DMY, MDY
	MDY, YMD
	MDY, DMY, YMD

https://en.wikipedia.org/wiki/List_of_date_formats_by_country



Colour	Order styles	Main regions and countries
	DMY	**Europe**: Italy, Netherlands, Turkey, Ireland, etc. **North America**: Mexico, various Caribbean islands **Central America**: Guatemala, Honduras, Panama, etc. **South America**: Brazil, Colombia, Chile, Argentina, Peru, Venezuela, etc. **North Africa**: Egypt, Algeria, Morocco, Tunisia, etc. **East Africa**: Somalia **West, Central, and Southern Africa**: Nigeria, Ethiopia, DRC, Tanzania, Sudan, Uganda, South Africa, etc. **West Asia**: Iraq, Saudi Arabia, Yemen, etc. **Central Asia**: Tajikistan, Kyrgyzstan, Turkmenistan **East and Southeast Asia**: Indonesia, Thailand, Cambodia, etc. **South Asia**: Pakistan, Bangladesh **Oceania**: Papua New Guinea, New Zealand, etc. **Middle East**: United Arab Emirates, Qatar, Oman, Yemen, Saudi Arabia, Kuwait, Iraq, Egypt, Syria, Lebanon, Israel, Jordan
	YMD	China, Japan, South Korea, Taiwan, Hungary, Mongolia, Lithuania, Bhutan
	MDY	Some US island territories
	DMY, YMD	India, Russia, Vietnam, Germany, Iran, United Kingdom, France, Myanmar, Spain, Poland, Uzbekistan, Afghanistan, Nepal, Australia, Cameroon, Sri Lanka, Malaysia, Singapore, etc.
	DMY, MDY	Philippines, Togo, Puerto Rico, Cayman Islands, Greenland
	MDY, YMD	United States
	MDY, DMY, YMD	Kenya, Canada, Ghana

The `lubridate` package

```
> now()
[1] "2025-09-03 15:54:54 HKT"
> now("GMT")
[1] "2025-09-03 07:54:56 GMT"
> OlsonNames() |> head(10)
 [1] "Africa/Abidjan"     "Africa/Accra"
 [3] "Africa/Addis_Ababa" "Africa/Algiers"
 [5] "Africa/Asmara"      "Africa/Asmera"
 [7] "Africa/Bamako"      "Africa/Bangui"
 [9] "Africa/Banjul"      "Africa/Bissau"
> now("Hongkong")
[1] "2025-09-03 15:54:59 HKT"
```

In R base, you can get the current time typing `sys.time()`.

In `lubridate`, you can use `now()`, which also permits you to define the time zone

```
> now() |> hour()
[1] 13
> now() |> minute()
[1] 26
> now() |> second()
[1] 53.60153
```

We can also extract hours, minutes, and seconds

The `lubridate` package

```
> x <- c("12:34:56")
> hms(x)
[1] "12H 34M 56S"
> x <- "Nov/2/2012 12:34:56"
> mdy_hms(x)
[1] "2012-11-02 12:34:56 UTC"
```

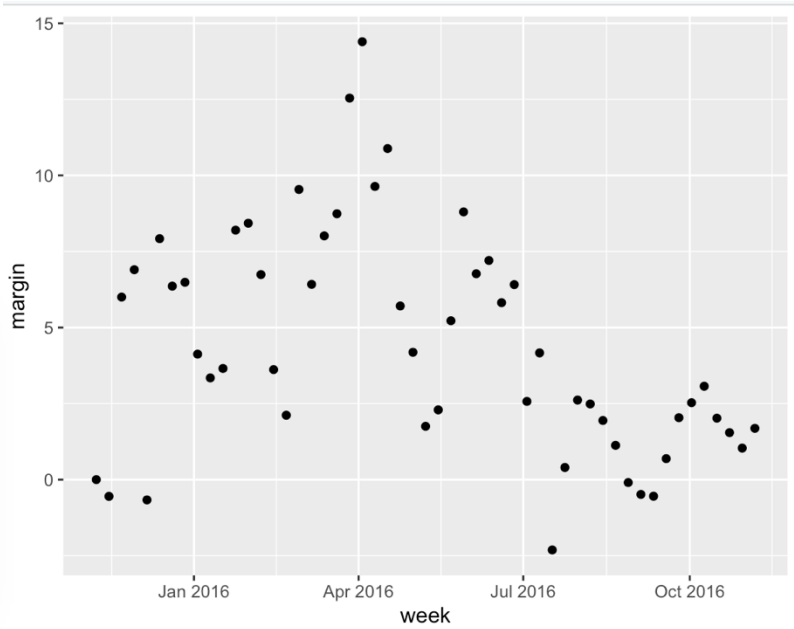
`hms, mdy_hms`

```
> make_date(2019, 7, 6)
[1] "2019-07-06"
```

`make_date`

`round_date` can round dates to nearest year, quarter, month, week, day, hour, minutes, or seconds.

```
> polls_us_election_2016 |>
+ mutate(week = round_date(startdate, "week")) |>
+ group_by(week) |>
+ summarize(margin = mean(rawpoll_clinton - rawpoll_trump)) |>
+ ggplot(aes(week, margin)) +
+ geom_point()
```



In this lecture

- String processing with `stringr`
  - `str_detect, str_replace, str_split`
- Regular Expression
  - `str_view_all`
  - Different syntaxes, wildcards, anchors, quantifiers
  - Groups
  - Search and replace with regex
- Parsing date and time
  - `lubridate`